Hazelcast: Soma Distribuída em um Cluster

Explicação do código e dos slides 14 a 18

Pedro Souza David Cristian Matheus Cardoso

Universidade Federal de Ouro Preto
BCC362 — Sistemas Distribuídos
Trabalho Prático 01

Como usar este material

Cada seção corresponde a um slide da apresentação (14 a 18). Para cada um há: o que o slide mostra, o trecho de código correspondente, a explicação linha a linha e uma **fala sugerida** para ser dita durante a apresentação. No final há um apêndice com o código de apoio à demonstração, os resultados medidos e perguntas prováveis da banca.

Visão geral do código

O projeto tem quatro arquivos Java em `src/`. Apenas dois contêm o algoritmo; os outros configuram o cluster ou servem de apoio à demonstração.

Arquivo	Papel	É o algoritmo?
`Main.java`	Ponto de entrada. Sobe um **Member** (configura cluster, rede, backup, executor) ou repassa o comando ao Cliente.	Não — configuração
`Cliente.java`	Conecta ao cluster de fora. Funções `carregar` (coloca 1..N no IMap) e `somar` (dispara a tarefa e junta os parciais).	**Sim** — carregar e somar
`SomaTask.java`	Tarefa enviada a cada membro. Soma apenas as chaves locais daquele membro.	**Sim** — núcleo
`EstadoTask.java`	Conta quantas partições primárias cada membro possui. Usada só na demonstração.	Não — apoio

Fluxo completo de uma execução

Linha do tempo

```
1. java Main membro 5701   (x3)  -> três JVMs sobem e formam o cluster "tp01-soma"
2. java Main carregar N          -> Cliente coloca 1..N no IMap "numeros" (hash -> partição -> membro)
3. java Main somar N             -> Cliente envia SomaTask a TODOS os membros
4. cada membro                    -> soma só suas chaves locais (localKeySet) e devolve o parcial
5. Cliente                        -> soma os parciais, confere com n(n+1)/2 e imprime VALIDADO
```

SLIDE 14

Aplicação do TP

O que o slide mostra: o problema (1 + 2 + 3 + ... + N) e o fluxo Cliente → IMap "numeros" → membros 5701/5702/5703 calculam somas → Cliente recebe. Este slide é a "ponte" entre a teoria (slides 1–13) e o código. Para explicá-lo, mostramos as três peças que montam esse fluxo.

14.1 — Os membros do cluster (Main.java, linhas 11–34)

Main.java — método membro(int porta)

```
Config config = new Config();
config.setClusterName(CLUSTER);                                    // "tp01-soma"
config.getNetworkConfig().setPort(porta).setPortAutoIncrement(false);
var join = config.getNetworkConfig().getJoin();
join.getMulticastConfig().setEnabled(false);
join.getAutoDetectionConfig().setEnabled(false);
join.getTcpIpConfig().setEnabled(true).setMembers(Arrays.asList(SEEDS.split(",")));

config.addMapConfig(new MapConfig("numeros").setBackupCount(1));
config.addExecutorConfig(new ExecutorConfig("soma").setPoolSize(2));
...
config.setProperty("hazelcast.heartbeat.interval.seconds", "1");
config.setProperty("hazelcast.max.no.heartbeat.seconds", "10");
HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
```

- **setClusterName**: só processos com o mesmo nome formam o mesmo cluster.
- **setPort(...) + setPortAutoIncrement(false)**: cada membro fica numa porta fixa (5701, 5702...), o que deixa claro na demo quem é quem.
- **Multicast/auto-detecção desligados, TCP/IP ligado**: a descoberta é feita por uma lista fixa de endereços (`127.0.0.1:5701..5704`). Cada membro tenta contatar os outros — é o "se encontram" do slide 8.
- **setBackupCount(1)**: cada partição do mapa `numeros` tem uma cópia em outro membro (slide 12).
- **ExecutorConfig("soma")**: cria o serviço de execução distribuída usado para rodar a SomaTask.
- **Heartbeat 1 s / 10 s**: um membro que fica 10 s sem responder é considerado morto — deixa a demonstração de falha rápida.
- As demais propriedades (threads) só limitam o consumo para rodar várias JVMs num notebook.

14.2 — O cliente conecta de fora (Cliente.java, linhas 97–101)

Cliente.java — executar(...)

```
ClientConfig config = new ClientConfig(); config.setClusterName(Main.CLUSTER);
config.getNetworkConfig().addAddress(Main.SEEDS.split(","));
config.getConnectionStrategyConfig().getConnectionRetryConfig()
      .setClusterConnectTimeoutMillis(30_000);
HazelcastInstance hz = HazelcastClient.newHazelcastClient(config);
```

O cliente **não guarda dados** e não participa do particionamento. Ele apenas se conecta usando o mesmo nome de cluster e recebe um `HazelcastInstance` com o qual acessa o IMap e o executor como se fossem objetos locais.

14.3 — Carga dos números no IMap (Cliente.java, linhas 11–19)

Cliente.java — carregar

```
static void carregar(HazelcastInstance hz, int n) {
    if (n < 0) throw new IllegalArgumentException("N deve ser não negativo");
    IMap<Integer, Long> mapa = hz.getMap("numeros");
    mapa.clear();
    Map<Integer, Long> entrada = new HashMap<>();
    for (int i = 1; i <= n; i++) entrada.put(i, (long) i);
    mapa.putAll(entrada);
    System.out.println("CARREGADO n=" + n);
}
```

- **hz.getMap("numeros")**: obtém o mapa distribuído — é a caixa "IMap numeros" do slide.
- **clear()**: apaga uma carga anterior, para não somar números antigos.
- Monta um `HashMap` local com chave *i* e valor *i* (1→1, 2→2, ...).
- **putAll**: envia tudo em lote. Para cada chave o Hazelcast calcula o **hash**, descobre a **partição** (entre 271) e grava no **membro dono** daquela partição, mais a cópia de backup.
- O programador não escolhe onde cada número fica — a distribuição é transparente (slide 11).

Fala sugerida: *"No slide 14 está o problema: somar de 1 até N. O fluxo tem três peças. Primeiro, cada membro é uma JVM configurada no Main, com o mesmo nome de cluster, porta fixa e backup 1. Depois, o cliente se conecta de fora. Por último, o cliente coloca os números 1 a N no IMap numeros com um putAll. A partir daí o Hazelcast aplica o hash em cada chave e espalha os números entre os membros — nós não controlamos onde cada um fica."*

SLIDE 15

Como a soma é executada? — SomaTask

O que o slide mostra: o laço que cada membro executa e os quatro passos: acessar o IMap, olhar as chaves primárias **locais**, calcular a soma parcial e devolvê-la. Esta é a parte central do trabalho.

SomaTask.java — arquivo completo

```
public class SomaTask implements Callable<Long>, Serializable, HazelcastInstanceAware {
    private static final long serialVersionUID = 1L;
    private transient HazelcastInstance hazelcast;

    public void setHazelcastInstance(HazelcastInstance instancia) {
        this.hazelcast = instancia;
    }

    public Long call() {
        IMap<Integer, Long> numeros = hazelcast.getMap("numeros");
        long soma = 0;
        // Cada membro soma somente as chaves primárias que pertencem a ele.
        for (Integer chave : numeros.localKeySet()) {
            Long valor = numeros.get(chave);
            if (valor == null) throw new IllegalStateException("Dado indisponível; repita a soma.");
            soma += valor;
        }
        return soma;
    }
}
```

15.1 — As três interfaces

Interface	Por que é necessária
`Callable<Long>`	É uma tarefa que **devolve um resultado** — a soma parcial do membro. (Um `Runnable` não devolveria nada.)
`Serializable`	O objeto SomaTask é **serializado e enviado pela rede** do cliente até cada membro. Ideia-chave: **o código vai até os dados**, e não os dados até o código.
`HazelcastInstanceAware`	Quando a tarefa chega no membro, o Hazelcast chama `setHazelcastInstance` e injeta a instância **local** daquele membro. Por isso o campo é `transient`: ele não viaja pela rede, é preenchido na chegada.

15.2 — O método call(), passo a passo

- **Passo 1 — acessa o IMap:** `hazelcast.getMap("numeros")` — o mesmo mapa que o cliente carregou.
- **Passo 2 — chaves locais:** `numeros.localKeySet()` devolve **apenas as chaves cuja partição primária pertence a este membro**. Cópias de backup não entram.
- **Passo 3 — soma parcial:** percorre essas chaves, lê o valor e acumula em `soma`.
- **Passo 4 — devolve:** `return soma` — o valor vira o resultado do Future que o cliente espera.

Por que localKeySet() garante a soma correta

Cada chave tem **exatamente um dono primário**. Então, somando só as chaves locais em cada membro: nenhum número é somado duas vezes (backup não conta) e nenhum número fica de fora (toda chave tem um dono). A soma dos parciais é, portanto, exatamente 1 + 2 + ... + N.

Proteção contra dado ausente

O `if (valor == null)` cobre o caso de uma partição estar sendo migrada no meio da soma (um membro caiu ou entrou). Em vez de devolver uma soma errada silenciosamente, a tarefa falha e o cliente refaz a soma inteira.

Origem: a classe é uma adaptação do exemplo `SumTask` da documentação oficial do Hazelcast 5.6 (Java Executor Service) — primeira referência do slide 18.

Fala sugerida: *"Esta é a parte central. A SomaTask é Serializable porque ela viaja pela rede até cada membro — o código vai até os dados. Quando chega, o Hazelcast injeta a instância local pelo HazelcastInstanceAware. O segredo está no localKeySet: cada membro pega só as chaves primárias dele. Como cada chave tem um único dono, nenhum número é somado duas vezes e nenhum fica de fora. Cada membro devolve sua soma parcial."*

SLIDE 16 Execução da tarefa — submitToAllMembers

O que o slide mostra: a linha que dispara a SomaTask em todos os membros e o diagrama SomaTask → 5701/5702/5703 (parciais) → Cliente. Aqui está o lado do cliente: distribuir a tarefa e juntar as respostas.

Cliente.java — somar (linhas 22–48)

```
static double somar(HazelcastInstance hz, int n) throws Exception {
    var membrosAntes = new HashSet<>(hz.getCluster().getMembers());
    long inicio = System.nanoTime();
    var respostas = hz.getExecutorService("soma").submitToAllMembers(new SomaTask());
    long total = 0;
    Map<String, Long> parciais = new TreeMap<>();
    try {
        for (var resposta : respostas.entrySet()) {
            long parcial = resposta.getValue().get(30, TimeUnit.SECONDS);
            total += parcial;
            parciais.put(resposta.getKey().getAddress().toString(), parcial);
        }
    } finally {
        for (var futura : respostas.values()) if (!futura.isDone()) futura.cancel(true);
    }
    double ms = (System.nanoTime() - inicio) / 1_000_000.0;
    long esperado = (long) n * (n + 1L) / 2;
    if (!membrosAntes.equals(new HashSet<>(hz.getCluster().getMembers())) || total != esperado)
        throw new IllegalStateException("Cluster mudou ou soma divergiu; repita após estabilizar.");
    System.out.println("PARCIAIS " + parciais);
    System.out.println("VALIDADO n=" + n + " soma=" + total
                       + " membros=" + membrosAntes.size() + " ms=" + ms);
    return ms;
}
```

16.1 — Distribuir (a linha do slide)

- `getExecutorService("soma")`: obtém o executor distribuído configurado no Main.
- `submitToAllMembers(new SomaTask())`: envia **uma cópia** da tarefa para **cada membro** do cluster. Todos executam **em paralelo**.
- Retorna um `Map<Member, Future<Long>>`: uma "resposta pendente" por membro. O cliente não fica bloqueado no envio.

16.2 — Juntar os parciais (redução)

- O laço percorre cada Future e chama `get(30, SECONDS)`: espera o parcial daquele membro (no máximo 30 s) e soma em `total`.
- Guarda cada parcial num `TreeMap` por endereço — é o que aparece no log como `PARCIAIS {[127.0.0.1]:5701=..., ...}`.
- O bloco `finally` cancela tarefas pendentes se alguma falhar — não há reenvio por pedaço.
- É um padrão **map/reduce** simples: cada membro faz o "map" (soma local) e o cliente faz o "reduce" (soma dos parciais).

16.3 — Validação do resultado

- Compara o total com a fórmula de Gauss: `n(n+1)/2`. Para N = 100 000 → 5 000 050 000.
- Compara o conjunto de membros **antes e depois**. Se mudou (alguém caiu ou entrou durante a soma), o resultado é **rejeitado**.

- Isso liga diretamente com o aviso do slide 12: **o backup recupera os dados, não a tarefa que estava rodando** no membro que caiu. A próxima rodada refaz a soma inteira.
- O tempo medido (`ms`) inclui envio, execução e recebimento — é o valor usado no benchmark.

Saída típica

```
PARCIAIS {[127.0.0.1]:5701=..., [127.0.0.1]:5702=..., [127.0.0.1]:5703=...}
VALIDADO n=100000 soma=5000050000 membros=3 ms=2418.80086
```

Fala sugerida: *"No cliente, uma única linha distribui o processamento: submitToAllMembers. Ela manda uma cópia da SomaTask para cada membro, que calculam em paralelo, e devolve um Future por membro. O cliente espera cada parcial e acumula. No fim conferimos com a fórmula n vezes n mais 1 sobre 2 e verificamos se o cluster mudou no meio — se mudou, o resultado é descartado, porque o backup salva os dados mas não a tarefa que estava em execução."*

SLIDE 17

Conclusão

O que o slide mostra: a definição final do Hazelcast (biblioteca Java que cuida de comunicação TCP, coordenação, particionamento por hash e tolerância a falhas) e a cadeia de conceitos HAZELCAST → CLUSTER → MEMBERS → CLIENT → IMAP → PARTIÇÕES → BACKUPS → EXECUÇÃO DISTRIBUÍDA → SOMATASK.

Cada conceito da cadeia no código

A tabela abaixo mostra onde cada conceito do slide aparece concretamente no projeto — útil para responder "onde isso está no código?".

Conceito	Onde está no código	O que faz
Hazelcast	`Hazelcast.newHazelcastInstance(config)` Main.java:31	Transforma a JVM comum num nó do sistema distribuído.
Cluster	`setClusterName("tp01-soma")` Main.java:13	Nome comum que agrupa os membros.
Members	`java Main membro 5701` Main.java:11–34	Cada JVM guarda dados e executa tarefas.
Client	`HazelcastClient.newHazelcastClient` Cliente.java:101	Acessa o cluster de fora, sem guardar dados.
IMap	`hz.getMap("numeros")` Cliente.java:13	Mapa distribuído com os números 1..N.
Partições	`mapa.putAll(entrada)` Cliente.java:17	Hash da chave decide a partição e o membro dono.
Backups	`setBackupCount(1)` Main.java:21	Uma cópia de cada partição em outro membro.
Execução distribuída	`submitToAllMembers(...)` Cliente.java:27	Mesma tarefa roda em todos os membros em paralelo.
SomaTask	`numeros.localKeySet()` SomaTask.java:20	Cada membro soma só a sua parcela dos dados.

O que NÃO precisamos implementar

Comunicação por sockets TCP, descoberta de membros, cálculo de hash e partições, replicação de backups, detecção de falha por heartbeat e redistribuição de partições. Tudo isso vem pronto do Hazelcast — o nosso código de algoritmo cabe em cerca de 40 linhas (SomaTask + carregar + somar).

Resultados que sustentam a conclusão

N	Membros	Soma (mediana)	Speedup S(p)	Eficiência E(p)
100 000	1	6 432 ms	1,00	1,00
100 000	2	2 922 ms	2,20	1,10
100 000	3	2 537 ms	2,54	0,85

Ressalva importante: os membros são JVMs no **mesmo computador**, e cada número passa pelo IMap (serialização, threads, memória). Por isso o ganho não é proporcional ao número de membros e não deve ser extrapolado para máquinas separadas. A eficiência acima de 1 com 2 membros é efeito de aquecimento/cache desta campanha, não um ganho físico superior a 100%.

Fala sugerida: *"No fim das contas, o Hazelcast é uma biblioteca Java que transforma processos comuns em um sistema distribuído. Todo o trabalho difícil — comunicação TCP, descoberta, particionamento por hash, backup e recuperação — vem pronto. Nosso código de soma tem umas 40 linhas. Com 100 mil números, a soma caiu de 6,4 segundos com um membro para 2,5 segundos com três. O objetivo do TP não era a soma em si, mas entender como dados e processamento são distribuídos entre vários processos."*

SLIDE 18 Referências

O que o slide mostra: as fontes usadas. Vale explicar rapidamente **para que** cada uma foi usada, pois mostra que as decisões do código têm base na documentação oficial.

Referência	Para que foi usada no trabalho
Hazelcast 5.6: Java Executor Service — exemplo SumTask	Origem direta da SomaTask (Callable + Serializable + HazelcastInstanceAware + localKeySet) e do uso de submitToAllMembers. Slides 15 e 16.
Hazelcast 5.6: Making Your Map Data Safe	Base do setBackupCount(1) e da explicação de que o backup protege os dados. Slide 12.
Hazelcast: Data Partitioning	Explica chave → hash → partição → membro e as 271 partições. Slides 7 e 11; confirmado nos logs (91/90/90).
Estudo de caso: BNP Paribas Bank Polska	Exemplo real em bancos: dados em memória para resposta rápida. Slide 5.
Estudo de caso: British Gas / Hive (IoT)	Exemplo real em IoT: estado compartilhado entre nós. Slide 5.
Hazelcast: Payment Processing Use Case	Contexto de uso em pagamentos. Slide 5.
Billion Events Per Second (Jet 4.3)	Benchmark histórico de escala (45 nós). Mostra o potencial, mas não é comparável ao nosso experimento.

Links completos em `docs/REFERENCIAS.md`.

> ***Fala sugerida:*** *"As referências principais são a documentação oficial: nossa SomaTask é adaptada do exemplo SumTask do Executor Service, e as configurações de backup e partição seguem as páginas Making Your Map Data Safe e Data Partitioning. Os estudos de caso mostram onde o Hazelcast é usado de verdade."*

Apêndice A — Código de apoio à demonstração ao vivo

Estas partes **não** fazem parte do algoritmo de soma; existem para mostrar ao vivo o que acontece por dentro do cluster (slide 13 e demonstração).

EstadoTask — quantas partições cada membro tem

EstadoTask.java

```
public String call() {
    long particoes = hazelcast.getPartitionService().getPartitions().stream()
        .filter(p -> hazelcast.getCluster().getLocalMember().equals(p.getOwner())).count();
    return "primarias=" + particoes
         + " seguro=" + hazelcast.getPartitionService().isClusterSafe();
}
```

- Mesma estrutura da SomaTask (Callable + Serializable + HazelcastInstanceAware), mas em vez de somar números, conta as partições cujo dono é o membro local.
- `isClusterSafe()` indica se todas as partições estão com backup em dia (nenhuma migração em curso).

Resultado real (logs 17-estado.log e 19-estado.log)

```
Com 3 membros:                          Depois que um membro caiu:
[127.0.0.1]:5701 primarias=91           [127.0.0.1]:5701 primarias=136
[127.0.0.1]:5702 primarias=90           [127.0.0.1]:5702 primarias=135
[127.0.0.1]:5703 primarias=90
       total = 271                             total = 271
```

As 271 partições foram redistribuídas automaticamente entre os membros restantes usando os backups — a prova concreta do slide 13.

Comandos de apoio no Cliente

Comando	O que faz
`estado`	Roda a EstadoTask em todos os membros e imprime as partições de cada um.
`pronto M`	Espera até o cluster ter M membros e estar "seguro" (sem migrações). Timeout de 90 s.
`acompanhar N`	Repete a soma a cada 2 s. Se o cluster estiver reorganizando, imprime AGUARDANDO; se a soma for interrompida, imprime RODADA_INTERROMPIDA e a próxima rodada refaz tudo.
`benchmark CSV`	Para N = 1 000, 10 000 e 100 000: carrega, faz 1 aquecimento e 3 medições, grava no CSV.

Trecho real do comando acompanhar durante falha e expansão

```
VALIDADO n=100000 soma=5000050000 membros=3 ms=2418.80086
RODADA_INTERROMPIDA ExecutionException; próxima rodada refaz a soma inteira
   ^ um membro foi derrubado no meio da soma
VALIDADO n=100000 soma=5000050000 membros=2 ms=3307.606102
   ^ dados recuperados pelos backups; soma correta com 2 membros
RODADA_INTERROMPIDA IllegalStateException; próxima rodada refaz a soma inteira
   ^ um membro novo entrou no meio da soma
VALIDADO n=100000 soma=5000050000 membros=3 ms=2779.31615
   ^ cluster expandido para 3 membros
```

Apêndice B — Perguntas prováveis e respostas curtas

P: Por que usar localKeySet() e não keySet()?

`keySet()` devolveria **todas** as chaves do cluster; cada membro somaria tudo e o total sairia multiplicado pelo número de membros. `localKeySet()` devolve só as chaves primárias locais, então cada número é somado exatamente uma vez.

P: Os backups não fazem um número ser somado duas vezes?

Não. `localKeySet()` considera apenas as partições **primárias** do membro; as cópias de backup são ignoradas.

P: Por que a SomaTask precisa ser Serializable?

Porque ela é enviada pela rede do cliente para cada membro. Sem serialização o Hazelcast não conseguiria transmitir o objeto.

P: Por que o campo hazelcast é transient?

A instância do Hazelcast não pode (nem deve) ser serializada. Ela é injetada no destino por `setHazelcastInstance`, graças ao `HazelcastInstanceAware`.

P: O que acontece se um membro cai no meio da soma?

Os **dados** sobrevivem graças ao backup e as partições são reorganizadas. A **tarefa** que rodava no membro morto se perde: o Future falha, o cliente rejeita a rodada e a próxima refaz a soma inteira.

P: Por que o ganho não é 3x com 3 membros?

Todas as JVMs estão no mesmo computador (disputam CPU e memória), e cada número passa pelo IMap com custo de serialização e acesso. Há também custo fixo de enviar tarefas e receber respostas.

P: Onde os números ficam guardados?

Na memória (heap) dos membros, divididos em 271 partições. Cada partição tem um dono primário e um backup em outro membro.

P: Como os membros se encontram?

Por TCP/IP com uma lista fixa de endereços (`127.0.0.1:5701..5704`). Multicast e auto-detecção foram desligados para o comportamento ser previsível.

P: Qual a diferença entre Member e Client?

Member guarda dados, executa tarefas e participa do particionamento. Client só se conecta de fora para ler/escrever dados e pedir operações.